

My Stadium

Gaddiel Esteban Nieto Rojas

Thomas Javier Mora Clavijo

Yeimy Daniela Pardo Mora

Frank Sebastián Ortegon Palomino

Universidad de Cundinamarca

Facultad de Ingeniería

Ingeniería en Sistemas y Computación

Fusagasugá

2026

Tabla de Contenido

Introducción	4
Planteamiento del Problema	5
Justificación	6
Pregunta del Problema.....	7
Objetivo General	8
Objetivos Específicos	8
Alcance	9
Limitaciones	9
Marco Teórico	10
Arquitectura de Microservicios.....	10
Sistemas de Ticketing y Gestión de Concurrencia.....	10
Race Conditions en Reserva de Asientos.....	11
Pasarelas de Pago Digitales.....	11
Códigos QR y Boletos Digitales.....	11
Seguridad en Aplicaciones Móviles de Pago	12
Marco Conceptual	13
Glosario de Términos	13
Análisis Comparativo.....	15
Investigación Académica Relacionada	15
Normas ISO y Estándares Aplicables.....	16
ISO/IEC 25010 — Calidad del Producto de Software	16
ISO/IEC 27001 — Seguridad de la Información.....	18
Seguridad en Datos de Pago.....	18
Accesibilidad	19
Ley 1581 de 2012 — Protección de Datos Personales (Colombia).....	19
Requerimientos del Sistema.....	20
Requerimientos Funcionales	20
Módulo de Autenticación (AUTH)	20
Módulo de Catálogo de Conciertos (CAT)	21
Módulo de Selección de Asientos (SEATS).....	22
Módulo de Compra y Pago (PURCHASE)	23
Requerimientos No Funcionales.....	24

Casos de Uso.....	26
Arquitectura de Microservicios	30
Visión General.....	30
Estrategia de Gestión de Asientos en Tiempo Real.....	30
Diagramas.....	32
Diagrama UML	32
Diagrama modelo-entidad-relación (MER).....	33
Diagrama de arquitectura	34
Plan de Pruebas.....	35
Tipos de Pruebas	35
Casos de Prueba Críticos.....	36
Plan de Proyecto y cronograma	38
Metodología de Desarrollo	38
Fases del Proyecto.....	38
Riesgos del Proyecto.....	40
Conclusiones.....	42
Referencias	43

Introducción

Actualmente, la compra de entradas para eventos deportivos y conciertos requiere de sistemas digitales que sean rápidos, seguros y fáciles de usar. Muchas plataformas presentan problemas relacionados con la organización del código, la dificultad a la hora de realizar mantenimientos y su poca escalabilidad cuando aumenta la cantidad de usuarios. Con el fin de ofrecer una solución eficiente, surge My Stadium, una aplicación diseñada para la gestión y venta de tickets de conciertos y eventos realizados en un estadio. El sistema permite administrar eventos, controlar la disponibilidad de entradas y facilitar el proceso de compra para los usuarios.

Además, el proyecto utiliza arquitectura limpia, una metodología de desarrollo que divide el sistema en capas independientes, permitiendo un código más ordenado, mantenible y adaptable a futuros cambios tecnológicos. Esto contribuye a mejorar la calidad del software y la experiencia tanto de los administradores como de los clientes de la plataforma.

Planteamiento del Problema

En la actualidad, la organización de conciertos y eventos en estadios requiere sistemas tecnológicos capaces de administrar grandes cantidades de usuarios y ventas de entradas en tiempo real. Sin embargo, muchos sistemas tradicionales presentan problemas como lentitud en las compras, mala organización de la información, fallos en la disponibilidad de tickets y dificultades para realizar mantenimientos o actualizaciones del software.

Además, algunos aplicativos no cuentan con una estructura de desarrollo adecuada, lo que provoca código desordenado, poca escalabilidad y errores frecuentes cuando el sistema crece o aumenta la demanda de usuarios. Estas dificultades afectan tanto a los administradores de los eventos como a los clientes, generando pérdidas de tiempo, problemas en la gestión de entradas y una mala experiencia de usuario.

Frente a esta situación, surge la necesidad de desarrollar una plataforma moderna y eficiente que permita gestionar conciertos y eventos deportivos de forma organizada y segura. Por ello, se propone el desarrollo de My Stadium, una aplicación de venta de tickets para estadios basada en arquitectura limpia, con el objetivo de optimizar la administración de eventos, mejorar el proceso de compra de entradas y garantizar un sistema más mantenible, seguro y escalable.

Justificación

El desarrollo de My Stadium es importante porque brinda una solución tecnológica eficiente para la gestión y venta de tickets de conciertos y eventos en estadios. Actualmente, la alta demanda de entradas y el crecimiento de los eventos masivos hacen necesario contar con plataformas rápidas, seguras y organizadas que faciliten tanto la administración como la experiencia de compra de los usuarios.

Este proyecto permitirá optimizar procesos como el registro de eventos, control de disponibilidad de entradas y gestión de compras, reduciendo errores y mejorando la atención al cliente. Además, al implementar arquitectura limpia, el sistema tendrá una mejor organización interna, permitiendo que el código sea más fácil de mantener, actualizar y escalar en el futuro.

Desde el punto de vista académico y profesional, el proyecto también aporta experiencia en el desarrollo de software moderno, aplicando buenas prácticas de programación y separación de responsabilidades entre capas. Esto contribuye a la creación de aplicaciones más robustas, reutilizables y eficientes.

En conclusión, My Stadium busca ofrecer una plataforma confiable para la venta de tickets, mejorando la gestión de eventos y proporcionando una experiencia más cómoda y segura para los usuarios.

Pregunta del Problema

¿Como desarrollar una aplicación de venta de tickets para eventos deportivos y conciertos en un estadio que permita gestionar eventos y compras de manera segura, eficiente y escalable mediante el uso de arquitectura limpia?

Objetivo General

Desarrollar una aplicación para la gestión y venta de tickets de eventos y conciertos en un estadio, utilizando arquitectura limpia para garantizar un sistema organizado, escalable, seguro y fácil de mantener.

Objetivos Específicos

Implementar un sistema de gestión de eventos y conciertos que permita registrar, actualizar y consultar información relacionada con artistas, fechas, horarios y disponibilidad de entradas.

Desarrollar un módulo de venta de tickets seguro y eficiente, permitiendo a los usuarios realizar compras, consultar asientos disponibles y gestionar sus reservas de manera rápida y confiable.

Aplicar arquitectura limpia en el desarrollo del proyecto, separando responsabilidades entre capas para mejorar la mantenibilidad, reutilización del código y escalabilidad del sistema.

Alcance

- El proyecto My Stadium comprende:
- Backend de microservicios con APIs REST y eventos en tiempo real via WebSockets
- Panel de administración web (React.js)
- Servicio de notificaciones push (Firebase Cloud Messaging)
- Servicio de generación y envío de PDF de boletos por email
- Sistema de caché distribuido para gestión de disponibilidad de asientos
- Base de datos relacional y no relacional para distintos dominios del sistema

Limitaciones

- La aplicación no contempla la reventa de boletos entre usuarios (marketplace secundario) en esta versión.
- No se incluye integración con sistemas de control de acceso físico al estadio (validación de QR en torniquetes) en la v1.0.
- El módulo de recomendaciones por género musical está excluido del alcance inicial.
- No se ofrece soporte para criptomonedas u otras pasarelas de pago distintas a MercadoPago.

Marco Teórico

Arquitectura de Microservicios

La arquitectura de microservicios es un estilo arquitectónico donde una aplicación se construye como un conjunto de servicios pequeños, independientes y desplegables de forma autónoma, cada uno ejecutándose en su propio proceso y comunicándose mediante mecanismos ligeros, típicamente APIs HTTP/REST o mensajería asíncrona.

Los principios fundamentales de los microservicios incluyen:

- Responsabilidad única: cada microservicio maneja un dominio de negocio específico.
- Descentralización: cada servicio gestiona sus propios datos y lógica de negocio.
- Resiliencia: el fallo de un servicio no debe causar el colapso de todo el sistema.
- Escalabilidad independiente: los servicios con mayor carga pueden escalarse horizontalmente sin afectar a otros.
- Despliegue independiente: los servicios pueden actualizarse sin coordinación compleja con otros equipos.

Sistemas de Ticketing y Gestión de Concurrencia

Los sistemas de venta de boletos son casos de estudio clásicos en ingeniería de software por el desafío de gestionar recursos finitos (asientos) bajo alta concurrencia.

Los problemas principales incluyen las race conditions y la consistencia eventual.

Race Conditions en Reserva de Asientos

Una race condition ocurre cuando dos o más transacciones concurrentes intentan reservar el mismo asiento simultáneamente. Sin mecanismos de control adecuados, ambas podrían completarse exitosamente, resultando en la doble venta del mismo asiento. Las estrategias para mitigar este problema incluyen el optimistic locking, el pessimistic locking con base de datos, y los sistemas de reserva temporal distribuida usando Redis con TTL. My Stadium implementa un sistema de reserva temporal de 10 minutos usando scripts Lua atómicos en Redis.

Pasarelas de Pago Digitales

MercadoPago, subsidiaria de MercadoLibre, es la pasarela de pago líder en Latinoamérica con presencia en 18 países. Ofrece múltiples métodos de pago adaptados al contexto regional: tarjetas de crédito y débito, PSE (Colombia), efectivo en puntos físicos (Efecty, Baloto), billetera digital MercadoPago, y financiación en cuotas. Su API REST ofrece Checkout Pro, Checkout API y Checkout Bricks.

Códigos QR y Boletos Digitales

Un código QR (Quick Response Code) es un tipo de código de barras bidimensional que puede almacenar información codificada legible por lectores ópticos. En el contexto del ticketing digital, los códigos QR funcionan como credenciales de acceso al evento, codificando información única de la transacción que puede verificarse offline una vez descargado el token de validación criptográfico.

Seguridad en Aplicaciones Móviles de Pago

La seguridad en aplicaciones de pago móvil involucra múltiples capas: comunicación encriptada (TLS 1.3), autenticación robusta (JWT con refresh tokens), validación de pagos del lado del servidor, protección contra inyecciones, y cumplimiento con PCI-DSS. My Stadium nunca almacenará datos de tarjetas de crédito directamente; toda la información sensible de pago es manejada exclusivamente por los servidores de MercadoPago.

Marco Conceptual

Glosario de Términos

Término	Definición
My Stadium	Nombre comercial de la aplicación móvil multiplataforma para compra de boletos de conciertos objeto de este proyecto.
Microservicio	Componente de software autónomo que implementa una capacidad de negocio específica, con su propio almacenamiento de datos y ciclo de vida de despliegue independiente.
API Gateway	Punto de entrada único al ecosistema de microservicios que enruta solicitudes, aplica políticas de seguridad y agrega respuestas.
JWT (JSON Web Token)	Estándar abierto (RFC 7519) para transmitir información de forma segura entre partes como un objeto JSON firmado digitalmente. Usado para autenticación stateless en My Stadium.
WebSocket	Protocolo de comunicación bidireccional y full-duplex sobre TCP, usado para actualizar la disponibilidad de asientos en tiempo real sin polling continuo.
Redis	Sistema de almacenamiento de datos en memoria de código abierto, usado como caché distribuida y para la gestión de reservas temporales de asientos mediante scripts Lua atómicos.
Docker	Plataforma de contenedorización que empaqueta aplicaciones y sus dependencias en unidades aisladas (contenedores) para garantizar portabilidad entre entornos.
Kubernetes (K8s)	Sistema de orquestación de contenedores que automatiza el despliegue, escalado y gestión de aplicaciones contenerizadas en los microservicios de My Stadium.
CQRS	Command Query Responsibility Segregation: patrón arquitectónico que separa las operaciones de escritura (comandos) de las de lectura (consultas) para optimizar cada una independientemente.

QR Code	Quick Response Code: código de barras bidimensional que codifica información legible por cámara, usado como boleto digital en My Stadium.
Boleto / Ticket	Representación digital de un derecho de acceso a un evento específico, asociado a un asiento particular o categoría de acceso general.
Mapa de calor	Representación visual que usa gradiente de colores (verde-amarillo-rojo) para indicar el nivel de ocupación de cada sección del estadio en tiempo real.
Reserva temporal	Estado transitorio de un asiento durante el proceso de compra, donde el asiento se bloquea por 10 minutos y se libera si la transacción no se completa.
MercadoPago	Pasarela de pago líder en Latinoamérica, subsidiaria de MercadoLibre, usada para procesar transacciones en My Stadium.
Flutter	Framework de desarrollo de UI de código abierto creado por Google, basado en el lenguaje Dart, para construir aplicaciones nativas compiladas para móvil, web y escritorio desde una única base de código.
Wallet Digital	Sección de la app donde el usuario accede a todos sus boletos comprados, pasados y futuros, con sus respectivos códigos QR funcionales offline.
ISO/IEC 25010	Norma internacional que define el modelo de calidad para productos de software, evaluando características como funcionalidad, fiabilidad, usabilidad, seguridad y mantenibilidad.
PCI-DSS	Payment Card Industry Data Security Standard: estándar de seguridad para organizaciones que manejan datos de tarjetas de pago.

Tabla 1. Glosario

Análisis Comparativo

Característica	Ticketmaster	StubHub	TuBoleta	Eventbrite	My Stadium
Mapa visual interactivo	✓	✓	Básico	X	✓✓
Mapa de calor por sección	X	X	X	X	✓
Reserva temporal con timer	✓	✓	X	X	✓
Boleto QR offline	✓	✓	Parcial	✓	✓
Wallet digital en app	✓	✓	✓	✓	✓
Pago adaptado a LATAM	Parcial	X	✓	Parcial	✓✓
Panel admin web	✓	✓	✓	✓	✓
Calidad UX/UI (1-5)	3.5	4	2.5	3.5	5 (meta)
Mercado objetivo	Global	Global	Colombia	Global	LATAM

Tabla 2. Analisis Comparativo

Investigación Académica Relacionada

Desde la perspectiva académica, diversas investigaciones han abordado los desafíos técnicos y de experiencia de usuario en sistemas de venta de boletos en línea.

Baymard Institute (2023), en su investigación sobre usabilidad en comercio electrónico, encontró que *"el proceso promedio de checkout en plataformas de e-commerce requiere 14.88 pasos, cuando podría completarse en 7 o menos"*. Este hallazgo respalda directamente el objetivo de My Stadium de completar la compra en menos de 5 pasos.

En el contexto colombiano, el informe del Banco Mundial (2021) sobre transformación digital señala que Colombia presenta una brecha significativa entre la penetración de smartphones (72%) y el uso efectivo de servicios de comercio electrónico (34%), indicando que la principal barrera no es el acceso a dispositivos sino la calidad de la experiencia de usuario en las aplicaciones disponibles.

Respecto a sistemas de tiempo real, Kleppmann (2017) en su obra sobre sistemas de datos a escala señala que *"la consistencia eventual en sistemas distribuidos es un compromiso inevitable cuando se priorizan la disponibilidad y la tolerancia a particiones"*. Esta observación fundamenta la elección de arquitectura CQRS con Redis para My Stadium: el estado de disponibilidad de asientos puede ser eventualmente consistente en lecturas de catálogo, pero debe ser fuertemente consistente en la operación de reserva.

Normas ISO y Estándares Aplicables

ISO/IEC 25010 — Calidad del Producto de Software

La norma ISO/IEC 25010:2011 (SQuaRE — System and Software Quality Requirements and Evaluation) define el modelo de calidad para sistemas y productos de software. My Stadium debe satisfacer las siguientes características de calidad:

Característica	Subcaracterísticas Aplicables	Aplicación en My Stadium
Adecuación Funcional	Complejidad, Corrección, Pertinencia	Todas las funciones de compra, selección de asientos y pago deben implementarse correctamente según los requisitos definidos.
Eficiencia de Rendimiento	Comportamiento temporal, Utilización de recursos, Capacidad	La app debe responder en < 2s para carga de catálogo y < 500ms para actualización de disponibilidad de asientos.
Compatibilidad	Coexistencia, Interoperabilidad	La app debe funcionar correctamente en iOS 15+ y Android 8+ simultáneamente sin conflictos.
Usabilidad	Reconocibilidad, Aprendibilidad, Operabilidad, Estética, Accesibilidad	Proceso de compra completable en < 5 pasos. Interfaz intuitiva sin manual de usuario. WCAG 2.1 AA.
Fiabilidad	Madurez, Disponibilidad, Tolerancia a fallos, Recuperabilidad	SLA de 99.9% de disponibilidad. Recuperación automática ante fallos de microservicios individuales.
Seguridad	Confidencialidad, Integridad, No repudio, Autenticidad	Cifrado TLS 1.3, JWT, hash de contraseñas (bcrypt), auditoría de accesos, firmas digitales en boletos.
Mantenibilidad	Modularidad, Reusabilidad, Analizabilidad, Modificabilidad	Arquitectura de microservicios con cobertura de pruebas > 80%. Documentación de APIs con Swagger.
Portabilidad	Adaptabilidad, Instalabilidad, Reemplazabilidad	Flutter garantiza portabilidad iOS/Android. Contenedores

		Docker para portabilidad del backend.
--	--	---------------------------------------

Tabla 3. Calidad del Producto

ISO/IEC 27001 — Seguridad de la Información

La norma ISO/IEC 27001:2022 establece los requisitos para un Sistema de Gestión de Seguridad de la Información (SGSI). Los controles aplicables a My Stadium incluyen:

- A.5 Políticas de seguridad: definición de política de privacidad y tratamiento de datos personales conforme a la Ley 1581 de 2012 (Colombia) y GDPR.
- A.9 Control de acceso: autenticación robusta con JWT, roles diferenciados (usuario, administrador, superadmin), principio de mínimo privilegio.
- A.10 Criptografía: TLS 1.3 para comunicaciones, bcrypt para contraseñas, AES-256 para datos sensibles en reposo.
- A.12 Seguridad en operaciones: logs de auditoría, monitoreo continuo con alertas, backups cifrados automáticos.
- A.14 Seguridad en el desarrollo: revisión de código (code review), pruebas SAST/DAST, gestión segura de secretos con HashiCorp Vault.
- A.17 Continuidad del negocio: plan de recuperación ante desastres con RPO < 1h y RTO < 4h.

Seguridad en Datos de Pago

El Payment Card Industry Data Security Standard (PCI-DSS v4.0) es el estándar de la industria de tarjetas de pago. Dado que My Stadium delega el procesamiento de datos de tarjetas a MercadoPago, el nivel de cumplimiento requerido es SAQ-A, el nivel más

sencillo, que aplica cuando el comercio externaliza completamente el manejo de datos de tarjeta.

Accesibilidad

Las Web Content Accessibility Guidelines (WCAG) 2.1, nivel AA, establecen criterios para hacer el contenido digital accesible a personas con discapacidades. My Stadium implementará contraste de color mínimo 4.5:1, soporte completo para lectores de pantalla (VoiceOver en iOS, TalkBack en Android), tamaños de toque mínimos de 44x44 puntos, y textos escalables sin pérdida de funcionalidad hasta 200% del tamaño base.

Ley 1581 de 2012 — Protección de Datos Personales (Colombia)

La Ley Estatutaria 1581 de 2012 de Colombia regula el tratamiento de datos personales. My Stadium debe obtener consentimiento explícito antes de recolectar datos personales, informar al titular la finalidad del tratamiento, garantizar los derechos ARCO (acceso, rectificación, cancelación y oposición), y registrar las bases de datos ante la Superintendencia de Industria y Comercio.

Requerimientos del Sistema

Requerimientos Funcionales

Módulo de Autenticación (AUTH)

ID	Nombre	Descripción	Prioridad
RF-001	Registro de usuario	El sistema debe permitir al usuario crear una cuenta con email y contraseña. El email debe ser único en el sistema.	Alta
RF-002	Verificación de email	Al registrarse, el sistema enviará un email de verificación. El usuario no podrá completar compras sin verificar su email.	Alta
RF-003	Inicio de sesión	El sistema debe autenticar usuarios con email y contraseña mediante JWT con token de acceso (15min) y refresh token (30 días).	Alta
RF-004	Recuperación de contraseña	El sistema debe permitir solicitar un enlace de recuperación de contraseña enviado al email registrado, válido por 1 hora.	Alta
RF-005	Cierre de sesión	El sistema debe permitir cerrar sesión, invalidando el refresh token activo del dispositivo.	Media

RF-006	Perfil de usuario	El usuario puede ver y editar su nombre y correo desde la sección de perfil.	Media
--------	-------------------	--	-------

Tabla 4. Auth

Módulo de Catálogo de Conciertos (CAT)

ID	Nombre	Descripción	Prioridad
RF-007	Listado de conciertos	La app mostrará un listado visual de conciertos próximos con imagen del artista, nombre, fecha, lugar y precio desde.	Alta
RF-008	Detalle del concierto	Al seleccionar un concierto, el usuario verá información completa: artista, descripción, fecha, hora, lugar y secciones disponibles.	Alta
RF-009	Mapa visual del estadio	El sistema mostrará un diagrama SVG interactivo del estadio dividido por secciones, con mapa de calor (verde/amarillo/rojo) según disponibilidad.	Alta
RF-010	Búsqueda y filtros	El usuario podrá buscar conciertos por nombre del artista o ciudad, y filtrar por fecha, precio y disponibilidad.	Media

RF-011	Estado del mapa de calor	Verde: > 60% disponible. Amarillo: 20%-60% disponible. Rojo: < 20% disponible. Las secciones agotadas se muestran en gris oscuro.	Alta
--------	--------------------------	---	-------------

Tabla 5. Catalogo

Módulo de Selección de Asientos (SEATS)

ID	Nombre	Descripción	Prioridad
RF-012	Vista de sección	Al seleccionar una sección del estadio, el sistema desplegará la grilla de asientos individuales de esa sección.	Alta
RF-013	Estado de asientos	Cada asiento se mostrará como: Disponible (verde), Reservado temporalmente (amarillo parpadeante), Ocupado (rojo).	Alta
RF-014	Selección de asientos	El usuario puede seleccionar entre 1 y 8 asientos por transacción. Al seleccionar, el asiento cambia a estado reservado temporalmente.	Alta
RF-015	Temporizador de reserva	Al seleccionar el primer asiento, inicia un contador regresivo de 10 minutos visible. Si	Alta

		expira, los asientos se liberan automáticamente.	Alta
RF-016	Actualización en tiempo real	El estado de los asientos se actualiza en tiempo real via WebSocket. Si otro usuario reserva el mismo asiento, se notifica y libera.	
RF-017	Resumen de selección	El sistema mostrará un resumen con los asientos seleccionados, sección, precio por asiento y total antes de proceder al pago.	

Tabla 6. Asientos

Módulo de Compra y Pago (PURCHASE)

ID	Nombre	Descripción	Prioridad
RF-018	Datos del asistente	Por cada asiento seleccionado, el usuario deberá ingresar nombre completo y fecha de nacimiento del asistente.	Alta
RF-019	Proceso de pago	El sistema redirigirá a MercadoPago Checkout Pro para procesar el pago con múltiples métodos: tarjeta, PSE, efectivo y billetera.	Alta
RF-020	Confirmación de compra	Tras el pago exitoso, el sistema generará un código de orden único, actualizará los asientos a vendido y notificará al usuario.	Alta

RF-021	Generación de QR	El sistema generará un código QR único por asiento, firmado criptográficamente, que se mostrará en el wallet digital del usuario.	Alta
RF-022	Email con PDF	El sistema enviará automáticamente un email con un PDF adjunto que contiene el resumen de la compra y los QR de todos los boletos.	Alta
RF-023	Manejo de pago fallido	Si el pago falla o es cancelado, los asientos reservados temporalmente se liberan inmediatamente y se notifica al usuario.	Alta

Tabla 7. Compra y pago

Requerimientos No Funcionales

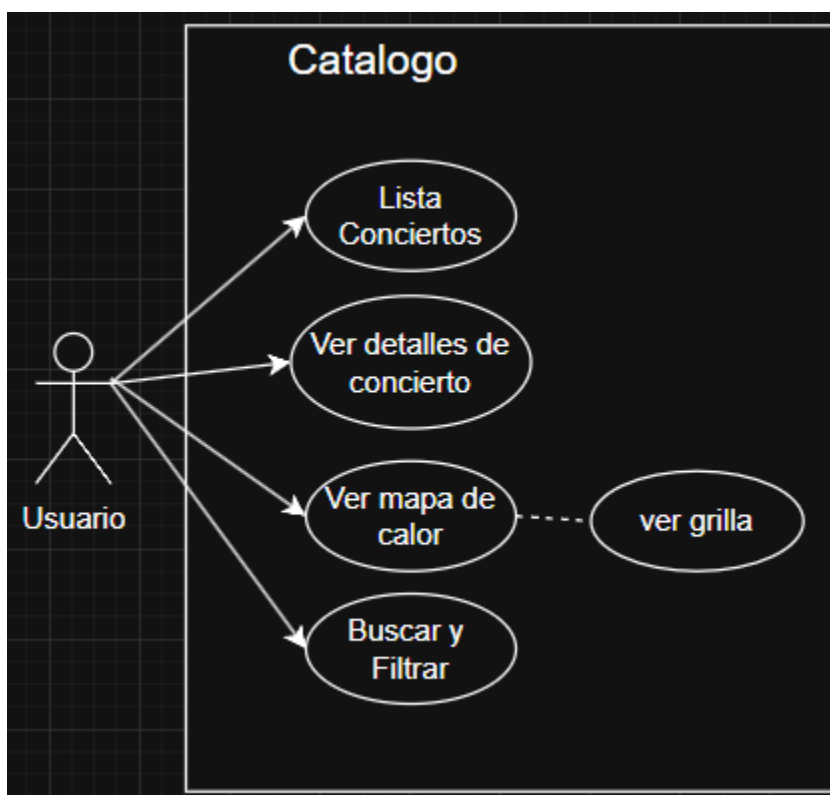
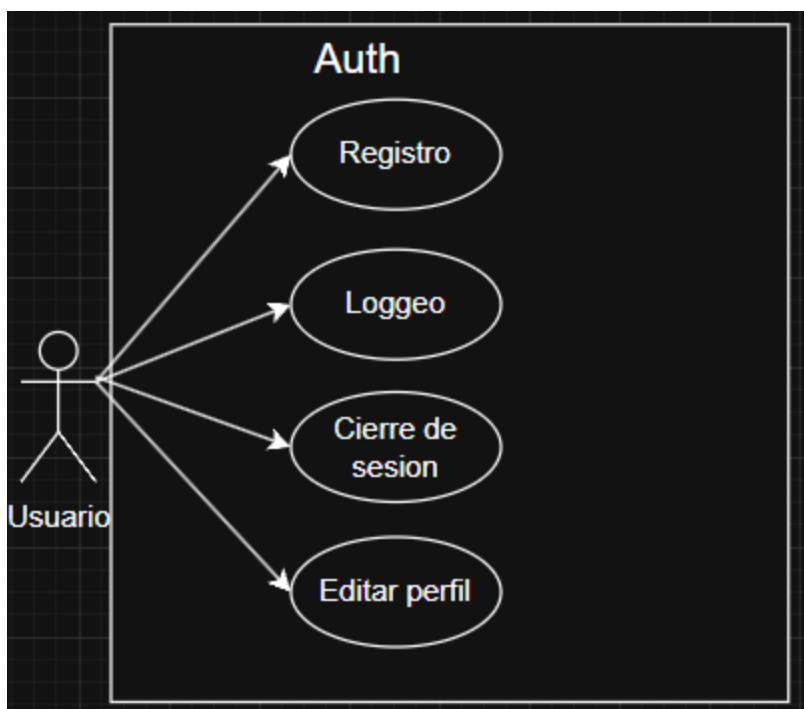
ID	Categoría	Requisito	Métrica de Aceptación
RNF-001	Rendimiento	Tiempo de carga del catálogo de conciertos	< 2 segundos en red 4G
RNF-002	Rendimiento	Tiempo de actualización de disponibilidad de asientos	< 500ms vía WebSocket
RNF-003	Rendimiento	Capacidad de usuarios concurrentes por evento	Mínimo 5.000 usuarios simultáneos sin degradación

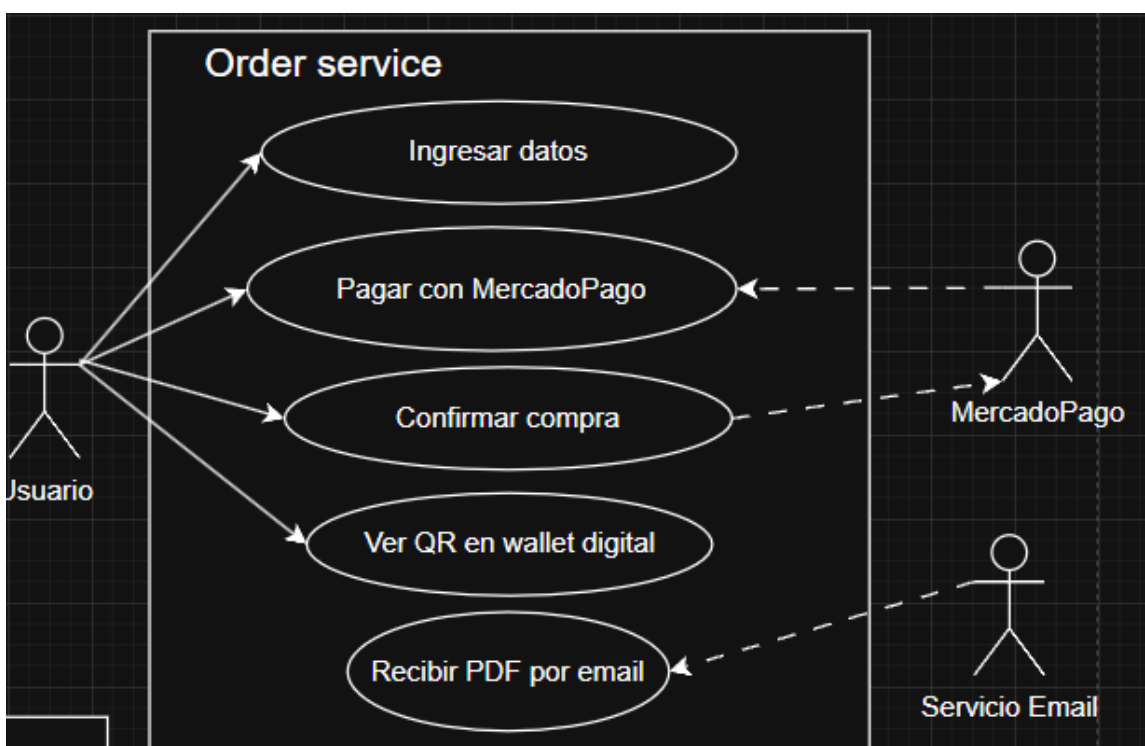
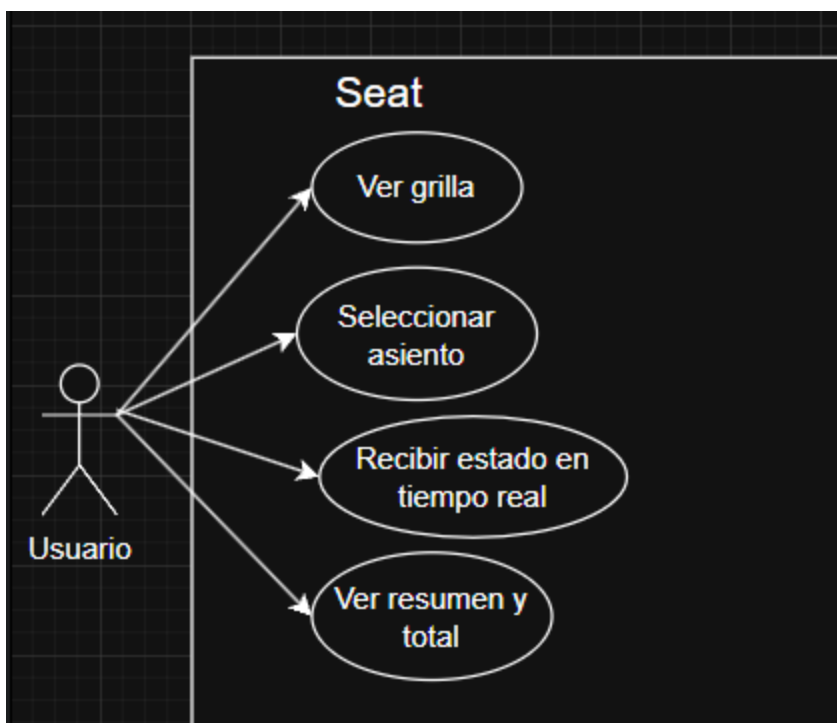
RNF-004	Disponibilidad	Uptime del sistema	99.9% mensual (SLA)
RNF-005	Seguridad	Protocolo de comunicación	TLS 1.3 obligatorio en todas las comunicaciones
RNF-006	Seguridad	Almacenamiento de contraseñas	bcrypt con cost factor mínimo 12
RNF-007	Usabilidad	Proceso de compra completo	Completable en máximo 5 pasos / 3 minutos
RNF-008	Compatibilidad	Versiones de SO soportadas	iOS 15.0+ y Android 8.0 (API Level 26)+
RNF-009	Mantenibilidad	Cobertura de pruebas	Mínimo 80% de cobertura de código en backend
RNF-010	Escalabilidad	Escalado horizontal	Escalar a 3x carga en < 5 minutos con Kubernetes HPA
RNF-011	Portabilidad	Tiempo de despliegue	Despliegue completo < 15 minutos con CI/CD pipeline
RNF-012	Accesibilidad	Cumplimiento WCAG	Nivel AA mínimo para todos los flujos críticos

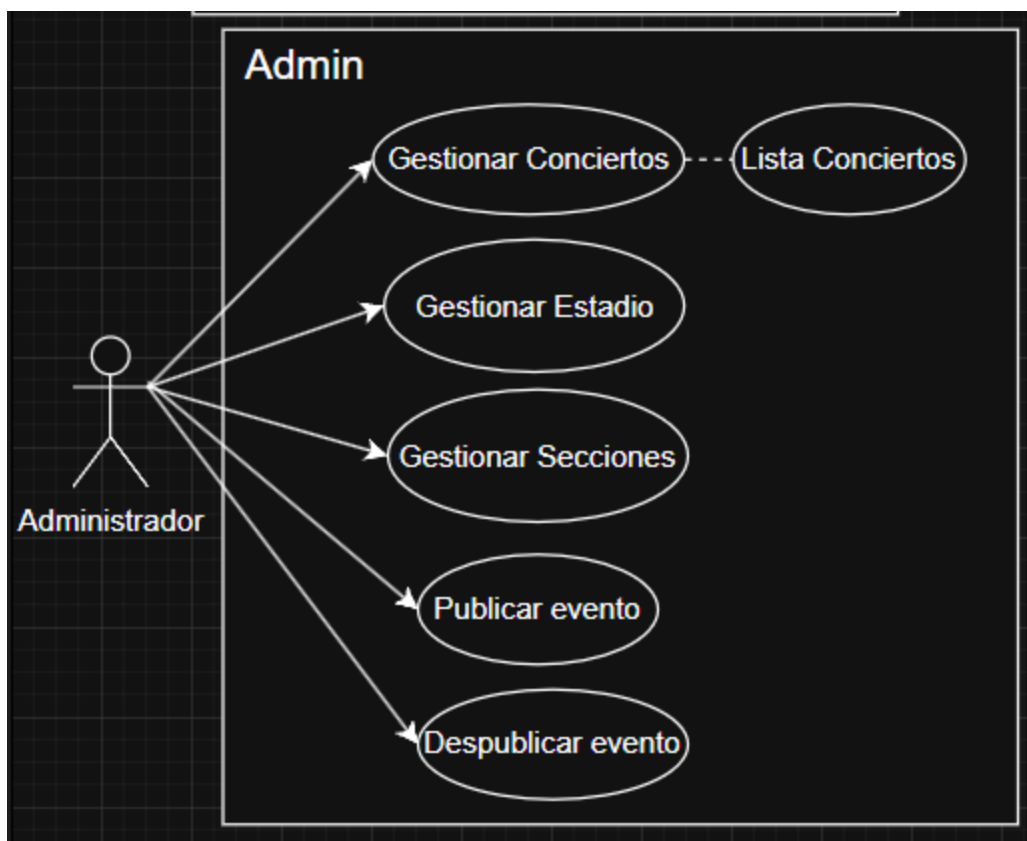
Tabla 8. Requerimientos No Funcionales

Casos de Uso

Actor	Descripción	Casos de Uso Principales
Usuario (Comprador)	Persona registrada en la app que desea comprar boletos para conciertos. Es el actor principal del sistema.	CU-001 al CU-005
Administrador	Operador del sistema con acceso al panel de administración web. Gestiona el catálogo de eventos y monitorea ventas.	CU-006 al CU-010
Sistema MercadoPago	Actor externo que procesa los pagos. Se comunica con el sistema mediante webhooks para notificar el resultado de las transacciones.	CU-004 (participante)
Sistema de Email	Actor externo que entrega los emails de confirmación con PDF a los usuarios.	CU-004 (participante)
Firebase FCM	Actor externo que entrega notificaciones push a los dispositivos móviles de los usuarios.	CU-004 (participante)







Arquitectura de Microservicios

Visión General

My Stadium adopta una arquitectura de microservicios con los siguientes principios: Domain-Driven Design (DDD) para la definición de contextos limitados, CQRS para optimizar el rendimiento de lectura/escritura, Event Sourcing para el historial completo de transacciones, y API-first para garantizar la interoperabilidad.

Microservicios Definidos

Servicio	Puerto	Responsabilidad	Base de Datos	Tecnología
Auth	8081	Registro, login, encriptación Bcrypt	Mystadium_auth	Spring boot + JPA + BCrypt
Catalogo	8082	CRUD de conciertos, imágenes de artistas	Mystadium_catalogo	Spring boot + JPA
Mapa de Calor	8084	Zona del estadio, mapa de calor, compra de boletos	Mystadium_mapa	Spring boot + JPA
Notifications	8083	PDF tickets, Envío de push notifications simuladas	(en memoria)	Spring boot + OpenPDF + ZXing
Frontend	80	Interfaz de usuario web	N/A	Spring boot + JPA + BCrypt

Tabla 9. Microservicios

Estrategia de Gestión de Asientos en Tiempo Real

La gestión de disponibilidad de asientos es el componente más crítico del sistema.

My Stadium implementa el siguiente patrón:

1. Escritura de reserva: el Seat Service ejecuta un script Lua atómico en Redis que verifica el estado actual y, si está disponible, actualiza a RESERVED_TEMP con TTL=600 en una operación atómica indivisible. Esto elimina la race condition.
2. Broadcast inmediato: tras la reserva exitosa, el Seat Service publica el cambio en un canal Redis Pub/Sub, retransmitido por WebSocket a los clientes en la room del concierto.
3. Persistencia eventual: el estado definitivo (SOLD) se escribe en PostgreSQL solo al confirmar el pago, garantizando durabilidad.
4. Recuperación de caídas: al reiniciar el Seat Service, reconstruye el caché de Redis desde PostgreSQL para los eventos activos.

Diagramas

Diagrama UML

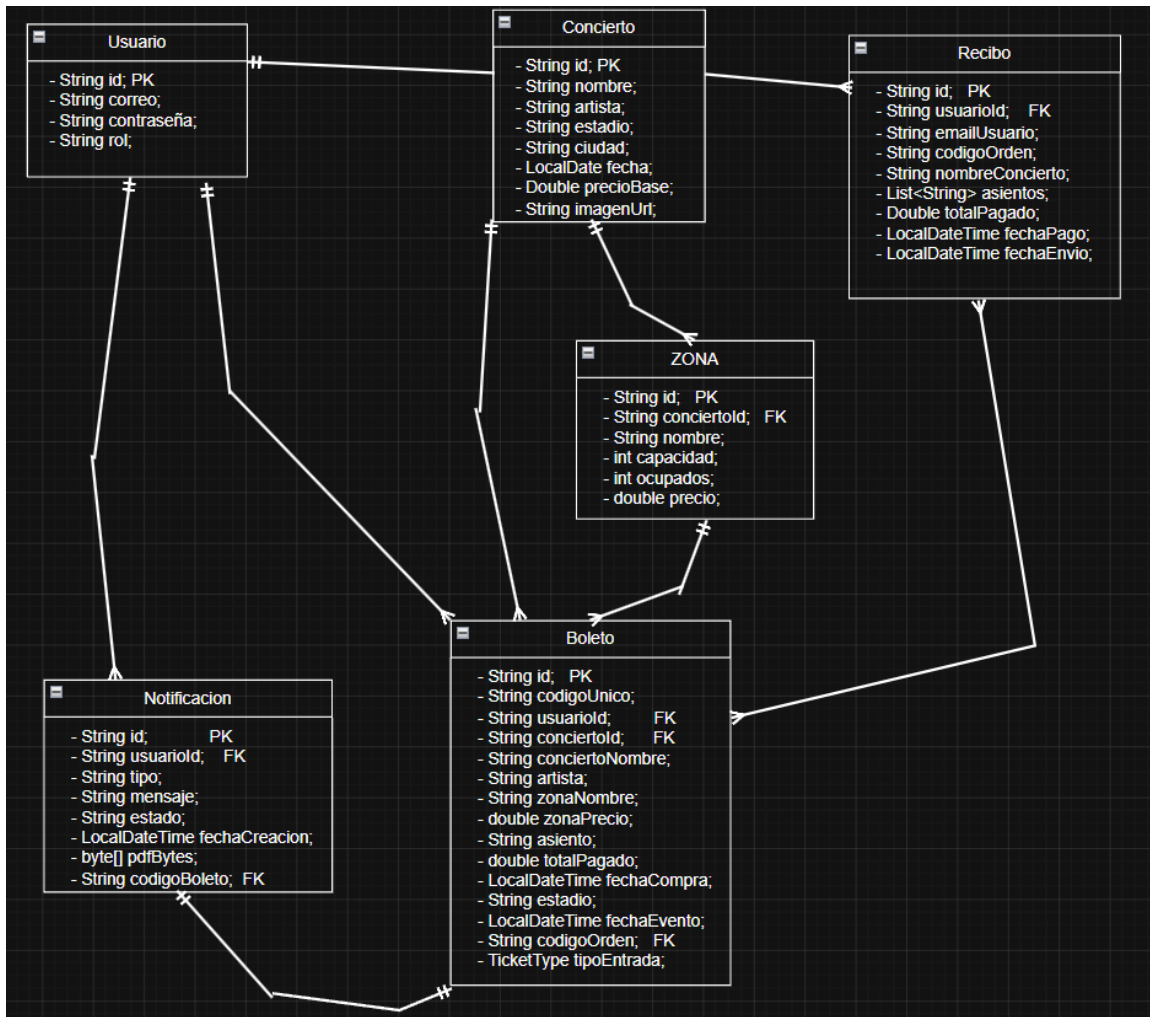


Diagrama modelo-entidad-relación (MER)

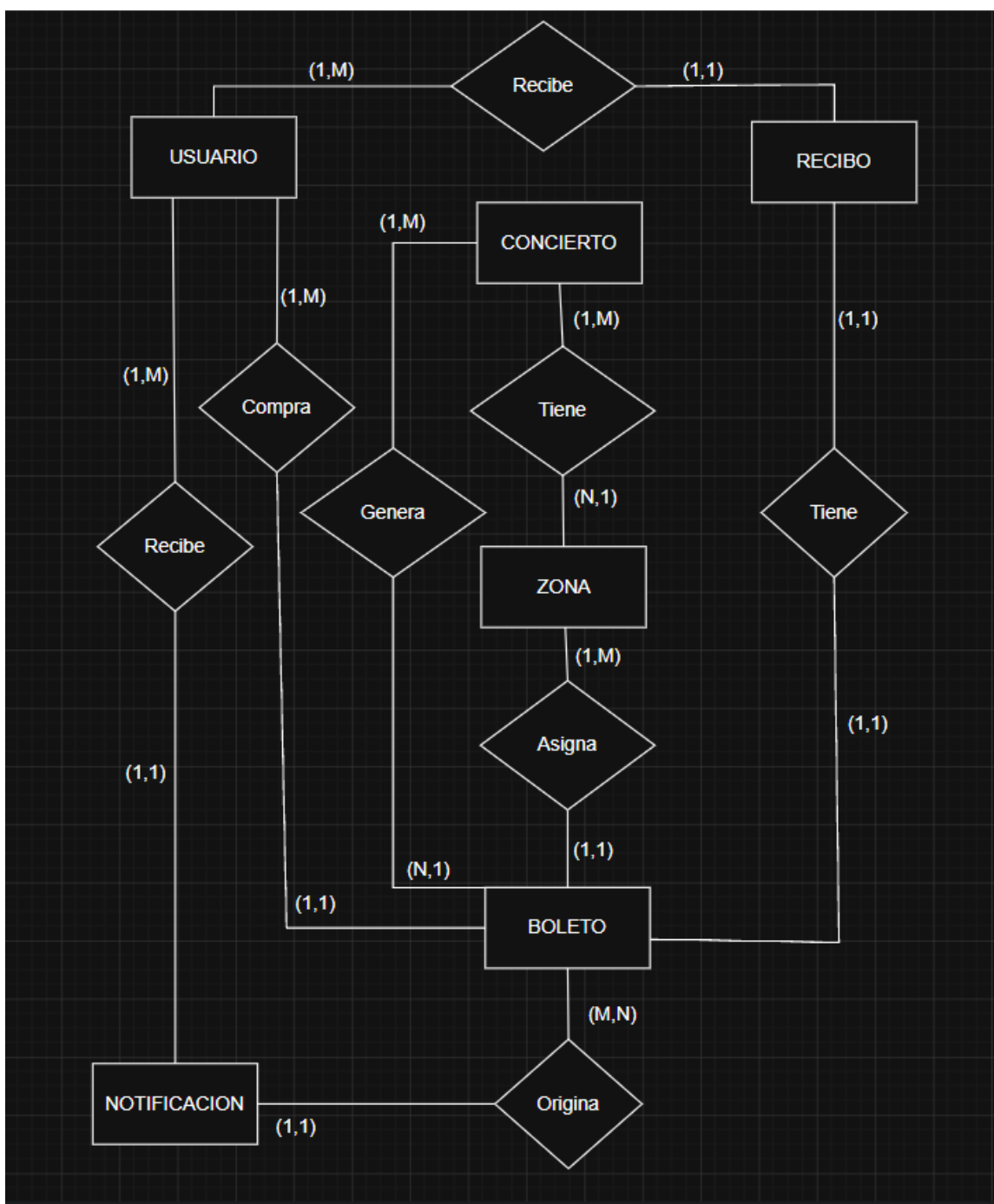
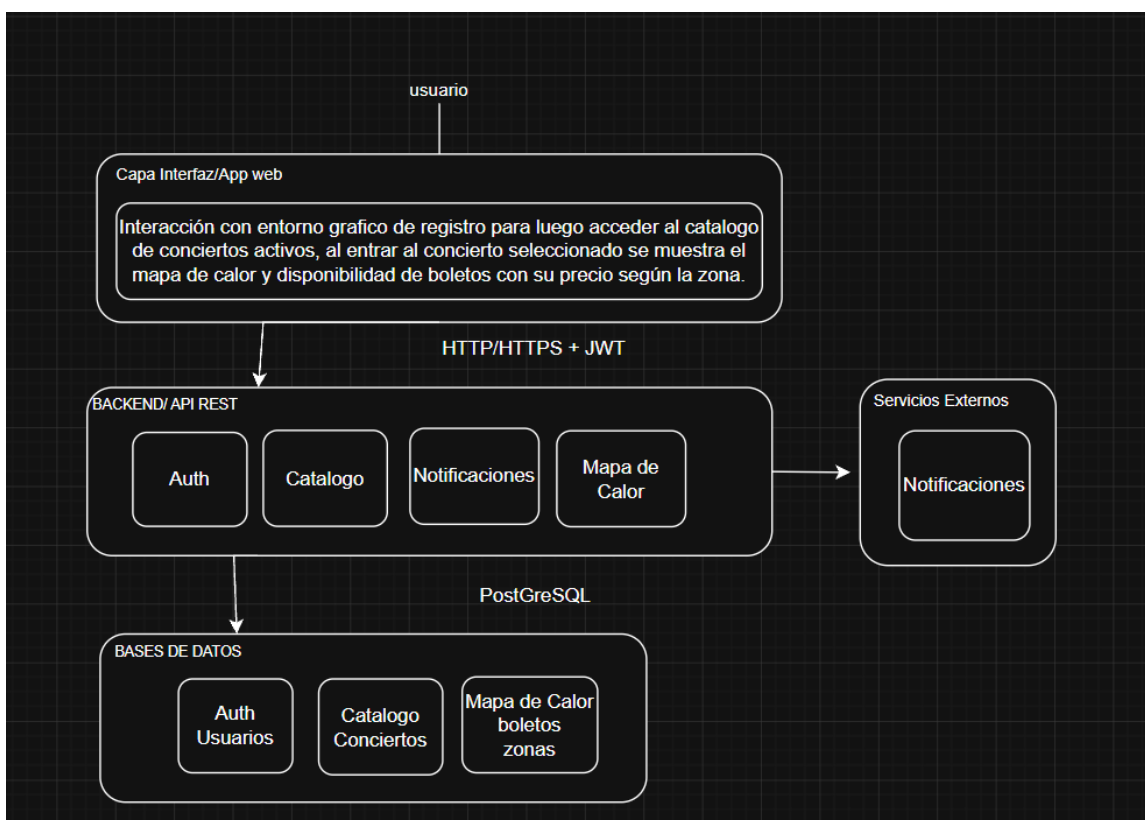


Diagrama de arquitectura



Plan de Pruebas

Tipos de Pruebas

Tipo	Herramienta	Alcance	Meta
Pruebas Unitarias	Jest (backend), Flutter test (frontend)	Lógica de negocio aislada: validaciones, cálculos de precios, generación de QR, scripts Lua de Redis.	> 80% cobertura
Pruebas de Integración	Jest + Supertest, TestContainers	Endpoints REST completos con BD real (PostgreSQL y Redis en Docker), flujo de webhooks de MercadoPago con sandbox.	> 70% de endpoints
Pruebas E2E	Flutter Integration Test	Flujos críticos: registro → login → compra → visualización de QR. Ejecutadas en dispositivos reales y emuladores.	5 flujos críticos 100%
Pruebas de Carga	Apache JMeter	Simular 5.000 usuarios concurrentes comprando boletos para el mismo evento. Medir latencia p95 y p99.	p95 < 2s bajo carga
Pruebas de Seguridad	OWASP ZAP, Snyk	Escaneo de vulnerabilidades OWASP Top 10, análisis de dependencias vulnerables, revisión de headers de seguridad.	0 vulnerabilidades críticas
Pruebas de Regresión	CI/CD Pipeline automático	Ejecución automática de suite completa en cada Pull Request antes de merge. Bloquea merge si alguna prueba falla.	100% suite verde en PR

Casos de Prueba Críticos

ID	Caso de Prueba	Pasos	Resultado Esperado	Tipo
TC-001	Doble reserva simultánea del mismo asiento	2 usuarios intentan reservar el mismo asiento en el mismo milisegundo	Solo uno logra la reserva. El otro recibe error 409.	Integración / Carga
TC-002	Expiración del temporizador de reserva	Usuario selecciona asiento y espera 10+ minutos sin pagar	Asiento se libera automáticamente en Redis. El usuario recibe notificación.	E2E
TC-003	Webhook de pago exitoso de MercadoPago	Simular POST webhook con status=approved	Asientos pasan a SOLD, QR generado, email enviado, wallet actualizado.	Integración
TC-004	Webhook de pago rechazado	Simular POST webhook con status=rejected	Asientos liberados, usuario notificado, orden en estado CANCELLED.	Integración
TC-005	Acceso offline al QR	Usuario descarga QR, activa modo avión, abre la app	QR visible sin conexión a internet.	E2E
TC-006	Login con credenciales incorrectas 5 veces	5 intentos de login con contraseña incorrecta	Cuenta bloqueada temporalmente 15 minutos. Email de alerta enviado.	Seguridad

TC-007	JWT expirado en solicitud	Request con token expirado → sistema debe renovar via refresh token	Renovación transparente. Si refresh también expiró, logout forzado.	Integración
TC-008	Carga de 5.000 usuarios concurrentes	JMeter: 5.000 VUs comprando simultáneamente	p95 < 2s, 0 errores 5xx, sin doble venta.	Carga

Plan de Proyecto y cronograma

Metodología de Desarrollo

My Stadium se desarrollará usando la metodología Scrum con sprints de 2 semanas. Los roles son: Product Owner (define prioridades de negocio), Scrum Master (facilita el proceso), equipo de desarrollo (desarrolladores frontend, backend, QA y DevOps).

Fases del Proyecto

Fase	Nombre	Duración	Entregables Principales	Sprint(s)
1	Inicio y Planificación	2 semanas	Documentación completa, backlog inicial priorizado, configuración del entorno de desarrollo, repositorios Git.	Sprint 0
2	Infraestructura Base	2 semanas	Docker Compose local, CI/CD pipeline básico en GitHub Actions, estructura de microservicios vacía, Kong configurado.	Sprint 1
3	Auth Service + App Base	2 semanas	Microservicio Auth funcional (registro, login, JWT). App Flutter con navegación base, pantallas de Login y Registro.	Sprint 2

4	Concert Service + Catálogo	2 semanas	Microservicio Concert Service. App: pantalla Home con catálogo de conciertos, búsqueda y filtros, detalle del concierto.	Sprint 3
5	Seat Service + Mapa Interactivo	3 semanas	Microservicio Seat Service con Redis y Lua. App: mapa SVG del estadio con mapa de calor, grilla de asientos, WebSocket, temporizador.	Sprint 4-5
6	Email + Wallet	2 semanas	Email Service con PDF. Pantalla Wallet con historial de boletos. Modo offline (almacenamiento local seguro).	Sprint 6
7	Panel Admin Web	2 semanas	Aplicación React.js del panel de administración: gestión de conciertos, estadios, asientos y dashboard de ventas.	Sprint 7
8	Pruebas y QA	3 semanas	Suite completa de pruebas unitarias, integración, E2E y de carga. Corrección de bugs. Auditoría de seguridad (OWASP ZAP).	Sprint 8-9

9	Pulimiento y Lanzamiento	2 semanas	Optimizaciones de rendimiento, pruebas de usabilidad, preparación para App Store y Google Play, despliegue en producción.	Sprint 10
---	--------------------------	-----------	---	-----------

Tabla 10. Fases del Proyecto

Duración total estimada: 22 semanas (~5.5 meses)

Riesgos del Proyecto

ID	Riesgo	Probab.	Impacto	Plan de Mitigación
R01	Picos de demanda en venta masiva de boletos	Alta	Alto	Pruebas de carga desde el Sprint 9. HPA en Kubernetes. Circuit breaker en Seat Service.
R02	Race conditions en reserva de asientos	Alta	Alto	Scripts Lua atómicos en Redis. Pruebas de concurrencia específicas (TC-001). Revisión de código obligatoria para el módulo Seat Service.
R03	Rechazo de la app por App Store / Google Play	Media	Alto	Revisión de políticas de tiendas antes del desarrollo. Pruebas en dispositivos reales.
R04	Vulnerabilidades de seguridad en pagos	Baja	Crítico	Nunca almacenar datos de tarjeta. Delegar a MercadoPago. Auditoría

				OWASP ZAP. Cumplimiento SAQ-A de PCI-DSS.
R05	Subestimación de la complejidad del mapa SVG interactivo	Alta	Medio	Prototipar el mapa en el Sprint 1 (spike técnico). Evaluar librerías SVG disponibles para Flutter.
R06				

Tabla 11. Riesgos del Proyecto

Conclusiones

El análisis y diseño de My Stadium permitió establecer una solución tecnológica orientada a la gestión y venta de tickets para conciertos y eventos en estadios. La propuesta desarrollada demuestra que es posible construir una plataforma organizada, segura y escalable capaz de atender las necesidades tanto de los usuarios como de los administradores de eventos.

La adopción de una arquitectura basada en microservicios y principios de arquitectura limpia favorece la separación de responsabilidades, la mantenibilidad del código y la escalabilidad de la plataforma. Esta estructura proporciona una base sólida para futuras ampliaciones del sistema, permitiendo incorporar nuevas funcionalidades sin afectar significativamente los componentes existentes.

Finalmente, el proyecto evidencia la importancia de aplicar buenas prácticas de ingeniería de software, estándares de calidad y mecanismos de seguridad desde las etapas iniciales de diseño, garantizando una solución alineada con las necesidades actuales de los sistemas de ticketing digital y los entornos de alta concurrencia.

Referencias

- Banco Mundial. (2021). *Transformación digital en Colombia: Brechas, oportunidades y recomendaciones de política*. World Bank Group.
- Congreso de Colombia. (2012). *Ley Estatutaria 1581 de 2012: Protección de Datos Personales*.
- Firebase by Google. (2024). *Firestore Cloud Messaging Documentation*. <https://firebase.google.com/docs/cloud-messaging>
- Google. (2024). *Flutter Documentation*. <https://flutter.dev>
- International Organization for Standardization. (2011). *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*.
- MercadoPago Developers. (2024). *API Reference — MercadoPago*. <https://www.mercadopago.com.co/developers>
- OWASP Foundation. (2021). *OWASP Top Ten 2021*. <https://owasp.org/www-project-top-ten>
- PCI Security Standards Council. (2022). *Payment Card Industry Data Security Standard (PCI DSS) Version 4.0*.
- Redis Ltd. (2024). *Redis Documentation*. <https://redis.io/docs>
- Scribd. (s.f.). *Client challenge*. Recuperado el 29 de mayo de 2026, de <https://es.scribd.com/document/632776025/code-complete-2nd-edition-v413hav-1-en-es-merged>
- World Wide Web Consortium (W3C). (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation.